

MettleQ: A Trustworthy Native Apple-Silicon Backend for Quantum Simulation

Technical report, version 0.2 - 18 July 2026

Abstract

MettleQ is a local quantum-circuit simulation framework focused on native Apple Silicon acceleration and normal Qiskit and PennyLane workflows. It combines an exact dense statevector engine, an experimental bounded matrix-product-state (MPS) engine, explicit memory preflight, inspectable execution planning, and hand-written Metal kernels selected through MLX. The project descends from the MIT-licensed Qupertino repository but has been substantially extended and renamed. This report describes the current architecture, the optimization that recovers legal layers from SDK scheduling, a radix-16 single-qubit Metal kernel, matched CPU/GPU experiments, numerical validation, safe capacity limits, and remaining limitations.

On an otherwise idle 14-inch MacBook Pro with Apple M3 Pro, 12 CPU cores, 18 GPU cores, and 36 GB unified memory, MettleQ crossed Qiskit Aer CPU statevector near 20 qubits for the tested depth-three full-state workload. From 22 through 29 qubits it was 2.74x to 4.55x faster than the fastest scheduled Aer CPU method and 16.01x to 21.18x faster than PennyLane Lightning. All 18 Qiskit and PennyLane state comparisons passed a global-phase-aligned $5e-6$ maximum amplitude threshold; the worst observed error was $4.02e-7$. A separate guarded scalar-output capacity campaign reached 29 qubits and refused 30 qubits before allocation. These results are workload- and machine-specific measurements, not universal performance claims.

1. Motivation and scope

Qiskit Aer and PennyLane Lightning provide excellent CPU simulation, while their established GPU acceleration is oriented primarily toward NVIDIA CUDA systems. Apple Silicon machines contain capable integrated GPUs and unified memory, but a normal Qiskit or PennyLane user has fewer native options for using that GPU. MettleQ targets this gap.

The intended architecture is deliberately asymmetric:

- Aer and Lightning remain CPU references and practical fallbacks.
- MettleQ is the native Apple-GPU backend when measured work amortizes GPU

dispatch.

- Selection considers circuit width, memory, requested precision, topology,

output contract, and whether approximation is allowed.

- MettleQ MPS remains experimental until its accuracy evidence and performance

are competitive.

The project optimizes complete SDK calls, not only favorable isolated kernels. It does not combine independent CPU and GPU times into a cooperative speedup.

2. Project lineage

MettleQ is maintained as a private fork of BoltzmannEntropy/Quptino. The original repository, authorship, MIT license, and benchmark history are retained in the main README. The fork adds native SDK adapters, adaptive planning, exact-state memory safeguards, recoverable MPS numerics, convergence reports, topology-aware routing, midpoint-MPO experiments, synchronized benchmarking, and MettleQ Studio.

3. Architecture

Qiskit circuits enter through a BackendV2 adapter and PennyLane programs enter through a registered device. Both translate operations to one canonical gate stream. The planner selects statevector or MPS and one numerical device. Dense statevector execution then performs capability-gated pattern recognition and dispatches either hand-written Metal kernels or the MLX compatibility path.

Trust boundaries are explicit:

- Dense allocation is refused before state construction when the projected working set violates device or policy limits.
- Custom Metal is selected only after platform, device, dtype, backend, qubit index, and environment-policy checks.
- Execution reports expose original and optimized operation counts, matched patterns, concrete kernels, expected launches, and synchronization points.
- MPS reports expose SVD drivers and fallbacks, discarded weight, norm error, canonical center, routing swaps, bond growth, and convergence status.

4. Dependency-preserving SDK scheduling

An SDK transpiler may emit a correct topological order that launches a two-qubit gate as soon as its wires are ready. This can interleave entanglers with otherwise parallel single-qubit work and hide a GPU-efficient layer. MettleQ reconstructs the per-wire dependency directed acyclic graph. Among ready operations it schedules one-qubit work before ready multi-qubit work. Operations sharing a wire retain their exact original order; only disjoint operations move.

For the benchmark circuit this turns 94 transpiled operations into eight structured operations: four per-qubit single-qubit layers and four affine permutations. The historical pair of dependency-constrained U3 operations is now consumed by the sparse custom Metal path rather than generic MLX.

5. Radix-16 single-qubit fusion

After scheduling, arbitrary single-qubit layers were still memory bound. The previous kernel applied two 2x2 matrices per full-state traversal. The current radix-16 kernel loads the 16 amplitudes associated with four adjacent state index bits, applies four potentially different 2x2 matrices while the values remain in registers, and writes the 16 results once. Pair and single kernels handle the one-to-three-qubit tail.

This reduces full-state traversals from approximately $n/2$ to $n/4$ for uniform and per-qubit-varying layers. It is general to arbitrary supported 2x2 gates; it is not specialized to one benchmark angle or circuit. Execution-plan launch accounting and memory checkpointing were updated with the kernel.

6. Experimental method

6.1 Hardware and software

- MacBook Pro model Mac15,7
- Apple M3 Pro, 12 CPU cores (6 performance and 6 efficiency)

- 18-core integrated Apple GPU, Metal 4
- 36 GB unified memory
- arm64 macOS
- Python 3.13.2
- MettleQ 0.2.0, MLX 0.32.0, NumPy 2.4.6
- Qiskit 2.5.0, Qiskit Aer 0.17.2
- PennyLane 0.45.1, PennyLane Lightning 0.45.0

The final campaign ran while unrelated workloads were stopped. A dedicated caffeinate process prevented sleep. Each large width used a fresh worker so that statevectors and allocator caches were released before the next width.

6.2 Full-state crossover contract

Each framework executes the same depth-three circuit containing parameterized RY/RZ layers, alternating nearest-neighbor CNOT layers, and one long-range CNOT layer. Qiskit uses the fastest measured Aer CPU method from statevector and MPS where MPS is scheduled. PennyLane uses lightning.qubit complex128. MettleQ is forced to its exact Apple-GPU statevector path. Transpilation is excluded; complete SDK execution and full-state return are included. There is one warmup and three measured calls per arm, summarized by the median.

6.3 Accuracy contract

Reference and candidate states are aligned by their global phase and compared in bounded chunks. A row is accepted only when the maximum amplitude error is at most 5e-6. Aer truncation is disabled. The benchmark records raw timing samples, environment versions, hardware facts, safety refusals, and errors.

7. Results

Qubits	Aer CPU / MettleQ GPU	Lightning CPU / MettleQ GPU	Worst state error
16	0.255x	1.445x	3.75e-7
18	0.627x	3.710x	3.98e-7
20	1.327x	11.097x	4.02e-7
22	3.160x	16.429x	3.35e-7
24	4.554x	18.713x	1.99e-7
26	4.256x	19.229x	2.60e-7
27	3.561x	18.186x	2.21e-7
28	2.833x	21.177x	1.22e-7
29	2.739x	16.005x	1.33e-7

A ratio above one favors MettleQ. Aer remains faster at 16 and 18 qubits, as expected when fixed GPU and adapter costs dominate. The measured Qiskit crossover is 20 qubits for this family. MettleQ beats Lightning throughout the measured range, with a larger margin after 20 qubits.

The current run should not be interpreted as a clean ablation against the older frozen run because unrelated load was removed at the same time as the radix-16 change. Absolute MettleQ time, matched current ratios, and a separate kernel microbenchmark are the appropriate inputs for future attribution.

Measured crossover and guarded capacity

MettleQ large exact-state crossover on Apple M3 Pro
same circuit and full-state contract; errors are global-phase aligned

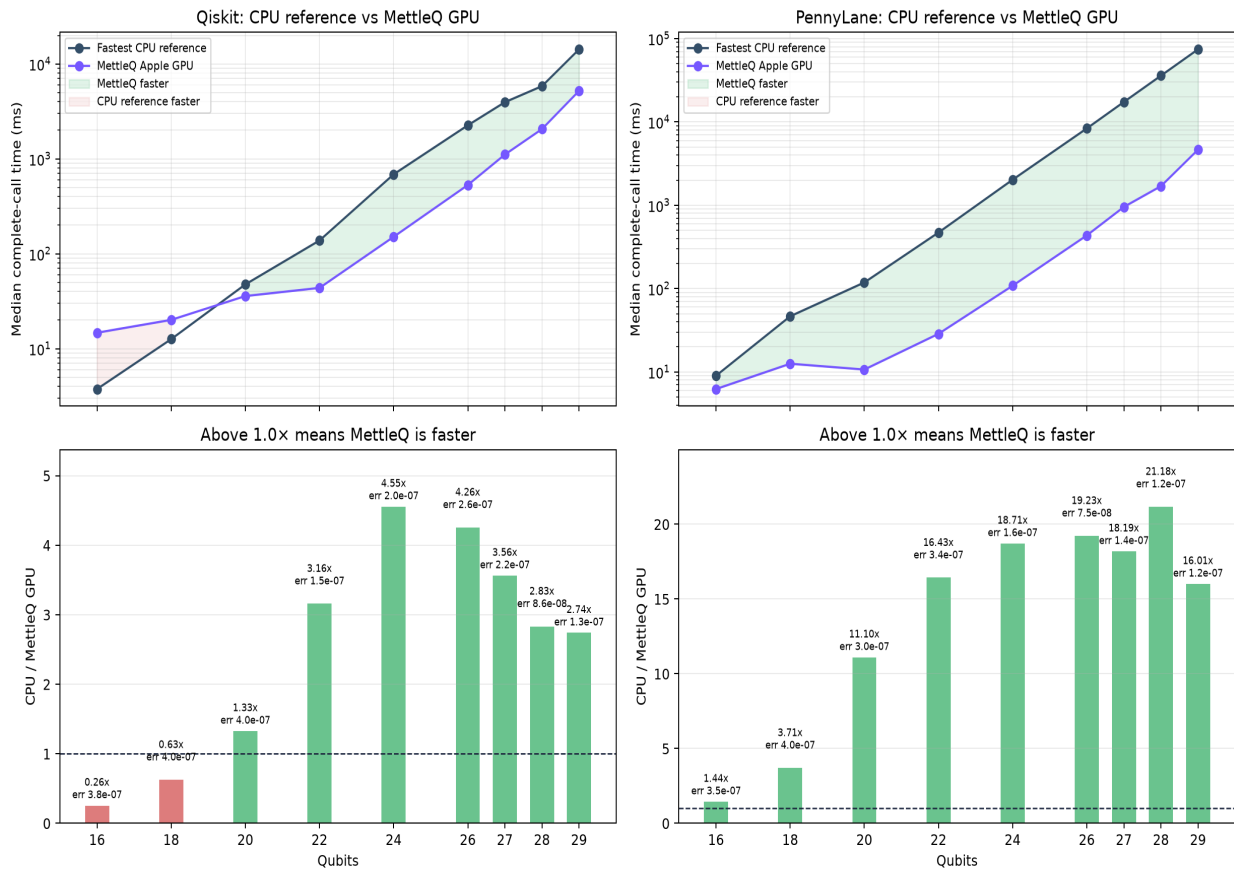


Figure 1. Complete full-state CPU/GPU crossover. Ratios above one favor MettleQ; annotations include maximum phase-aligned state error.

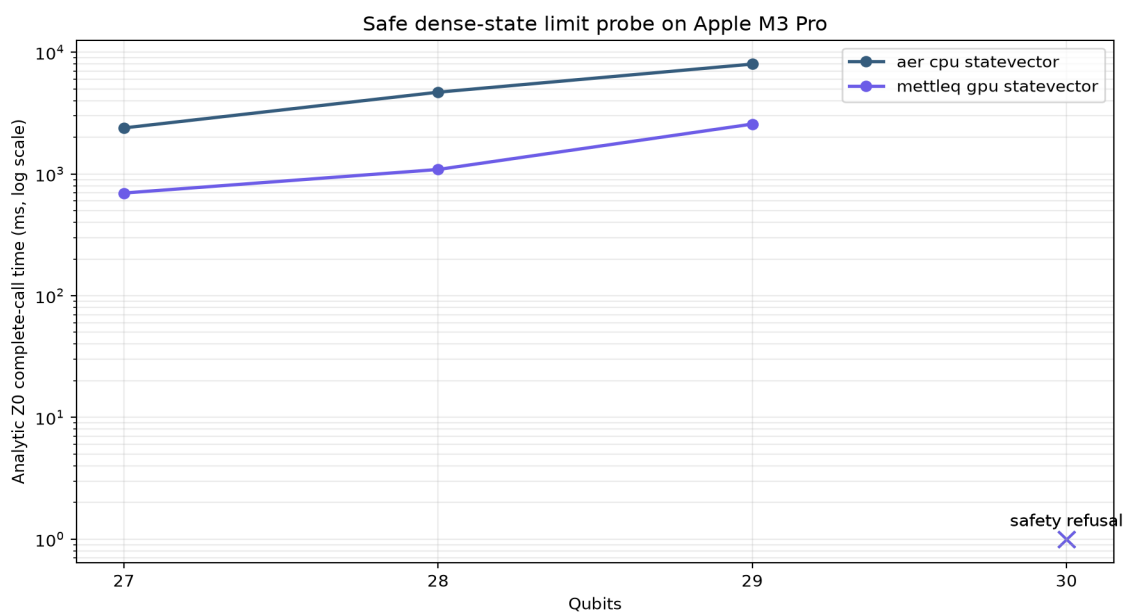


Figure 2. Monitored scalar-output capacity probe. The 30-qubit marks are pre-launch safety refusals.

8. Capacity and safety

A dense n -qubit state requires 2^n complex amplitudes. One complex64 state requires 8×2^n bytes; one complex128 state requires 16×2^n bytes, before temporary arrays and result copies. At 29 qubits these are 4 GiB and 8 GiB. At 40 qubits a complex128 state alone would require 16 TiB.

The scalar-output campaign uses a 45 percent physical-memory peak cap, 6 GiB pre-launch headroom, a 5 GiB runtime termination threshold, MLX cache disabled, fresh processes, and timeouts. It completed 27-29 qubits and refused 30:

Qubits	Aer CPU ms	MettleQ GPU ms	Aer / MettleQ	Absolute Z0 error
27	2388.091	696.868	3.427x	6.76e-10
28	4696.781	1088.176	4.316x	6.76e-10
29	7994.548	2569.355	3.111x	2.99e-7
30	refused	refused	-	-

This is a capacity result for a shallow scalar-output contract, not a claim that arbitrary 29-qubit circuits are safe.

9. Precision

MLX supports complex64 on GPU but does not provide a native GPU float64 path, so MettleQ does not claim GPU complex128 execution. Complex64 halves dense state storage and memory traffic relative to complex128 and is central to the observed capacity and speed. The cost is reduced precision. MettleQ therefore uses double-precision CPU references, explicit tolerances, phase-aligned state checks, and higher-precision host reductions where practical.

10. MPS status

The matched analytic-Z campaign completed 100-qubit GHZ and 30-40-qubit line, rainbow, and random long-range circuits. Aer CPU MPS was much faster than the current MettleQ CPU MPS implementation. Line circuits passed local accuracy thresholds, while rainbow and random long-range circuits exceeded the $D_{\max}=64$ discarded-weight threshold because routing and entanglement increased bond growth. These completed runs must not be called trustworthy without higher bond convergence.

The next MPS work is CPU contraction/SVD profiling, topology-aware routing, compact observable contracts, and only then GPU-resident contractions large enough to amortize transfers and launch overhead.

11. Validation and reproducibility

The release passes 372 automated tests and all 29 paired tutorial notebooks. The tutorial correctness audit reports nine bit-for-bit matches and 20 tolerance-based matches; every declared check passes. Frozen benchmark folders contain JSON, CSV, plots, protocol metadata, raw timing samples, and refusal records. The main README contains reproduction commands.

12. Limitations and future work

The most direct next improvements are:

- Build calibrated SDK dispatch that can select Aer or Lightning as the small CPU fallback and MettleQ as the Apple-GPU path without changing user code.
- Add circuit-family calibration based on gate mix, fusion coverage, expected launches, precision, and output contract rather than width alone.
- Profile radix-16 occupancy, register pressure, memory bandwidth, and tail

kernels with Metal counters on multiple M1-M4 machines.

- Add more native kernels for controlled arbitrary unitaries, multi-control gates, reductions, sampling, and expectation values so full-state host materialization is not required.
- Improve Qiskit result construction and state return to minimize avoidable host copies.
- Optimize MPS CPU contractions and SVDs before revisiting GPU MPS.
- Expand precision policies, accumulated-error studies, randomized circuit differential testing, and long-depth norm/error monitoring.
- Establish continuous performance regression testing with thermal state, power mode, OS version, and background-load metadata.

13. References

- MettleQ repository: <https://github.com/MonitSharma/MettleQ>
- Original Qupertino repository: <https://github.com/BoltzmannEntropy/Qupertino>
- Qiskit Aer simulator documentation: https://qiskit.github.io/qiskit-aer/stubs/qiskit_aer.AerSimulator.html
- Qiskit Aer MPS tutorial: https://qiskit.github.io/qiskit-aer/tutorials/7_matrix_product_state_method.html
- MLX documentation: <https://ml-explore.github.io/mlx/>
- MLX data types: https://ml-explore.github.io/mlx/build/html/python/data_types.html
- Apple Metal recommendedMaxWorkingSetSize:
<https://developer.apple.com/documentation/metal/mtldevice/recommendedmaxworkingsetsize>
- PennyLane Lightning: https://docs.pennylane.ai/projects/lightning/en/stable/lightning_qubit/device.html

Citation

If this technical report is used before a formal archival release, cite the repository, version, commit, benchmark folder, hardware profile, and access date. A DOI-backed release and archival citation remain future work.